PHENIKAA UNIVERSITY

PHENIKAA SCHOOL OF COMPUTING

COURSE MODULE: SOFTWARE ARCHITECTURE

PROJECT TITLE: ELECTRONICS E-COMMERCE WEBSITE

Course Class :	Software Architecture-1-2-25 (N01)
Instructor :	M.Sc. Vũ Quang Dũng
Group :	16
Group Members :	Đặng Việt Anh - 23010689

GROUP 16 — RESPONSIBILITY TABLE

Name	Student ID	Responsible Part
Nguyễn Tuấn Thiền	23010571	1. Introduction & Executive Summary 2. Project Requirements & Goals 3. Architectural Design (Use Case, C4 Model) 4. Monolithic Architecture 5. Microservices Architecture & Implementation 6. API Gateway, EDA, Circuit Breaker, Saga
Đặng Việt Anh	23010689	1. Testing & Verification 2. Deployment, ATAM & Conclusion

PREFACE

In the context of the rapid development of information technology, the application of software systems to management and service delivery has become an inevitable trend across many areas of modern society. Particularly in the electronics e-commerce sector, the increasing demand for fast, convenient, and seamless online shopping requires software systems not only to provide correct functionality but also to ensure scalability, reliability, high performance, and long-term maintainability.

The Software Architecture course plays an important role in equipping students with systematic thinking at the architectural level, including decomposing systems into independent components, selecting appropriate architectural styles, defining communication mechanisms among services, and optimizing quality attributes such as scalability, availability, and maintainability. Through this course, students are exposed to modern architectural approaches, especially Microservices Architecture combined with Event-Driven Architecture, which enables the development of flexible and sustainable large-scale systems in real-world environments.

Based on these practical needs, our team selected the topic "BUILDING AN ELECTRONICS E-COMMERCE WEBSITE — ShopSphere Platform" for this course project. The project focuses on requirement analysis (ASRs), overall architectural design using the C4 Model, service decomposition into independent microservices with Database-per-Service isolation, inter-service communication mechanisms (API Gateway, Transactional Outbox, Redis Queue), and system deployment using Docker containers.

Although we have made considerable efforts to complete this project, due to limitations in time, experience, and research scope, shortcomings are unavoidable. Therefore, we sincerely look forward to receiving valuable feedback and suggestions from our instructor to further improve the quality of this project.

ACKNOWLEDGEMENTS

We would like to express our sincere gratitude to ThS. Vũ Quang Dũng, our lecturer for the Software Architecture course, for his dedicated guidance, valuable feedback, and professional support throughout the completion of this project.

We would also like to thank the School of Computing — Phenikaa University for providing favorable learning conditions, facilities, and academic resources that enabled us to conduct research and complete this project effectively.

Finally, we would like to thank our friends and classmates for their support, discussions, and constructive suggestions during the learning and development process. Their encouragement has been a significant motivation for us to complete our tasks.

Despite our best efforts, the project may still contain limitations and mistakes. We highly appreciate any comments and recommendations from our instructor to help us improve and gain better experience for future studies and projects.

Mục lục

Contents

4. VERIFY THE SOLUTION (Test Outputs).....16

4.1. Testing Strategy.....16

4.1.1. Functional Test Coverage (Feature Tests).....16

4.1.2. Unit Test Coverage (Architecture Pattern Tests).....19

4.1.3. Functional Requirements to Test Mapping.....21

4.2. Test Results and Evidence.....23

4.3. ASR Verification Checklist.....26

4.4. User Interface Screenshots.....27

4.5. Terminal & Log Evidence (Lab 01–08).....30

Lab 06: API Gateway — 401/403/200 Tests (from Lab report).....30

Lab 06: Kong Direct API + Failure Handling (from Lab report).....32

Lab 06: Circuit Breaker Tests.....34

Lab 07: EDA Unit Tests — 15 passed (from Lab report).....34

Lab 07: Redis Consumer Log (from Lab report).....35

Lab 07: CQRS & Saga Pattern Tests.....36

Lab 08: Full Test Suite — All 221 Tests Passed.....36

5. JUSTIFY THE DECISIONS (ATAM, Rationale).....38

5.1. UML Deployment Diagram.....38

5.2. ATAM Quality Attribute Scenarios.....39

5.3. Architecture Trade-Off Analysis.....40

5.4. Conclusion & Reflection.....41

5.4.1. Conclusion.....41

5.4.2. Reflection 41

5.4.3. Future Directions 41

REFERENCES 43

LIST OF TABLE

Table 1.1: Customer Functional Requirements 10
Table 1.2: Administrator Functional Requirements 11
Table 2.1: Non-Functional Requirements 11
Table 2.2: ASR Scenarios 12
Table 3.1: Use Case Modeling 13
Table 4.1: Microservices Decomposition 28
Table 4.2: Communication Strategy Analysis 28
Table 6.1: Test Coverage Summary (25 Test Files) 40
Table 6.2: ASR Verification Checklist 42
Table 7.1: ATAM Quality Attribute Scenarios 46
Table 7.2: Architecture Trade-Off Analysis 47

LIST OF DIAGRAM

Hình 2.20: Biểu đồ tuần tự — EDA: Outbox Pattern + Redis Queue

Hình 2.19: Biểu đồ tuần tự — Circuit Breaker Pattern

Hình 2.18: Biểu đồ tuần tự — CQRS Pattern

Hình 2.17: Biểu đồ tuần tự — API Gateway Flow

Hình 2.8b: Make Purchase Critical Path (<<include>>/<<extend>>)

Hình 2.0: Biểu đồ ngữ cảnh hệ thống (System Context Diagram)

LIST OF FIGURE

Hình 3.1: Giao diện trang quản trị (Admin Dashboard) 56
Hình 3.2: Giao diện trang chủ người dùng (Customer Homepage) 56
Hình 3.3: Giao diện trang sản phẩm (Product Listing) 57
Hình 3.4: Giao diện trang bài viết (Article Page) 57
Hình 3.5: Giao diện giỏ hàng (Shopping Cart) 57
Hình 3.6: Giao diện đặt hàng (Checkout / Payment) 58
Hình 4.5a. Terminal — curl → HTTP 401 Unauthorized

Hình 4.5b. Terminal — curl Bearer wrong-token → HTTP 401

Hình 4.5c. Terminal — curl POST → HTTP 403 Forbidden

Hình 4.5d. Terminal — curl admin-token → HTTP 200 OK

Hình 4.6a. Terminal — curl /api/products via Kong → 200 OK

Hình 4.6b. Terminal — curl /api/health → healthy

Hình 4.6c. Terminal — docker stop → 504 Gateway Time-out

Hình 4.7. Terminal — Circuit Breaker Tests (18 passed)

Hình 4.8. Terminal — EDA Unit Tests: 15 passed (Lab 07)

Hình 4.9. Terminal — Redis Consumer Log (Lab 07)

Hình 4.10. Terminal — CQRS & Saga Tests (31 passed)

Hình 4.11. Terminal — All 221 Tests Passed (Lab 08)

Diagram 4.1. UML Deployment Diagram

ID	Description	Source File(s)	Priority
FR-01	Browse products by category, keyword, price	Modules/Catalog/CatalogController.php	High
FR-02	View product detail with images, ratings	Modules/Catalog/ProductDetailController.php	High
FR-03	Register/login with Customer auth guard	Modules/Customer/AuthUserController.php	High
FR-04	Manage shopping cart: add, update, remove	Modules/Cart/CartController.php	High
FR-05	Checkout: COD, VNPay, MoMo, PayPal, VietQR	Modules/Payment/PaymentController.php	High
FR-	Submit	Modules/Review/Rating	Medium

06	product reviews and ratings	Controller.php	
FR-07	Manage wishlist	Modules/Customer/WishlistController.php	Medium
FR-08	Contact support via form	Modules/Support/ContactController.php	Low
FR-09	Read articles and promotional content	Modules/Content/ArticleController.php	Low
ID	**Description**	**Source: Modules/Admin/**	**Priority**
FR-10	CRUD products with images, categories	AdminProductController.php	High
FR-11	Manage categories (hierarchical)	AdminCategoryController.php	High
FR-12	Process orders, view transactions	AdminTransactionController.php	High
FR-13	Manage user accounts	AdminUserController.php	Medium

FR-14	Manage articles and banners	AdminArticleController.php, AdminBannerController.php	Medium
FR-15	Manage suppliers	AdminSupplierController.php	Medium
FR-16	Manage warehouse: import goods	AdminWarehouseController.php, AdminImportGoodsController.php	Medium
FR-17	Moderate customer reviews	AdminRatingController.php	Low
FR-18	View/respond to contacts	AdminContactController.php	Low

Quality Attribute	Requirement	Implementation Evidence
Scalability	5,000 concurrent users; services independently scalable	docker-compose.microservices.yml: separate containers per service
Security	HTTPS, bcrypt passwords, dual auth guards, Gateway token	config/auth.php: 'admins' guard; GatewayTokenMiddleware.php: 401 response
Modifiability	New payment gateway = change Modules/Payment/ only	config/circuit_breaker.php: per-provider settings with fallbacks
Availabilit	>99% uptime; no lost	OutboxMessage model; notification-

y	order confirmations	service/src/RedisConsumer.php
Performance	Product pages < 500ms at P95	CQRS: ProductQueryService with Elasticsearch fallback to DB

ASR	Quality	Requirement	Architectural Decision
ASR-1	Scalability	Scale 500–5,000 users during peak	DB-per-service: mysql-catalog(3310), mysql-order(3311), mysql-user(3312). Kong distributes traffic.
ASR-2	Security	Admin via 'admins' guard; Gateway rejects 401	config/auth.php dual guards. GatewayTokenMiddleware validates Bearer token.
ASR-3	Modifiability	Payment failures must not cascade	CircuitBreaker.php: MoMo threshold=3 (fallback:qrcode), VNPay=5 (fallback:cod), PayPal=5 (fallback:cod).

Use Case	Actor	Main Flow	Source
Browse Products	Customer	1. Open homepage 2. Display featured products 3. Click category/search 4. Return filtered list	Modules/Catalog/ CatalogController
Make Purchase	Customer, Payment GW	1. Review cart 2. Enter shipping info 3. Select payment	Modules/Payment/ PaymentController Listeners/SaveOrder

		(COD/VNPay/MoMo/PayPal/QR) 4. Gateway processes 5. Webhook callback 6. OrderPlaced event → Outbox → Redis notification		PlacedToOutbox
Manage Products	Administrator	1. Login via /admin (CheckLoginAdmin middleware) 2. Navigate to Product Management 3. CRUD with images via AdminProductController 4. CQRS: ProductCommandService fires events		Modules/Admin/AdminProductController Services/CQRS/
Service	**Container**	**Port**	**Database**	**Key Patterns**
Catalog Service	catalog_service	9005	mysql-catalog:3310 (catalog_db)	CQRS, DB-per-service
Order Service	order_service	9002	mysql-order:3311 (order_db)	Saga, Outbox

User Service	user_service	9003	mysql-user:3312 (user_db)	Auth guards
Notification Service	notification_service	9004	— (consumes Redis)	BRPOP consumer
Kong API Gateway	kong_gateway	9000/9001	PostgreSQL (kong_db)	Routing, rate limiting
RabbitMQ Broker	rabbitmq_broker	5672/15672	—	Pub/Sub messaging
Consul Discovery	consul_discovery	8500	—	Health checks, registry
Jaeger Tracing	jaeger_tracing	16686	—	Distributed tracing
Prometheus	prometheus	9090	—	Metrics collection

Grafana	grafana	3000	—	Dashboard visualization

Activity	Protocol	Type	Source Evidence
Browse Products	HTTPS REST	Sync	Kong routes to catalog-service:9005
Place Order	HTTPS + Outbox	Sync+Async	SaveOrderPlacedToOutbox listener → OutboxMessage
Payment Processing	HTTPS Webhook	Sync+Async	PaymentController + Circuit Breaker middleware
Email Notification	Redis BRPOP	Async	notification-service/src/RedisConsumer.php
Service Registry	HTTP Consul	Sync	app/Services/ServiceDiscovery/ConsulClient.php

4. VERIFY THE SOLUTION (Test Outputs)

4.1. Testing Strategy

The testing strategy follows a layered approach aligned with the system's architecture: Feature Tests (6 files) verify end-to-end functional requirements through HTTP request/response cycles, while Unit Tests (19 files) verify individual architectural components in isolation. All 25 test files are located in the tests/ directory and can be executed via PHPUnit. The tests are organized to cover all 3 ASRs (Scalability, Security, Modifiability) and all 10 Functional Requirements (FR-01 through FR-10).

4.1.1. Functional Test Coverage (Feature Tests)

Feature tests simulate real HTTP requests to the application endpoints and verify the complete request-response lifecycle including middleware, controllers, models, and database interactions. Below are the 6 feature test files with their detailed test scenarios and FR mappings:

Test File: Feature/UserAuthenticationTest.php (10 test cases — maps to FR-02)

This test file verifies the complete user authentication lifecycle. test_login_page_loads_successfully: GET /login → asserts HTTP 200 and view 'auth.login'. test_register_page_loads_successfully: GET /register → asserts HTTP 200 and view 'auth.register'. test_user_cannot_login_with_invalid_credentials: POST /login with wrong email/password → asserts redirect (302) back to login with error message. test_user_registration_requires_valid_data: POST /register with empty name, invalid email, short password → asserts validation error (302/422). test_user_can_register_with_valid_data: POST /register with valid u_name, u_email, u_password → asserts redirect to login (302) indicating successful registration. test_duplicate_email_registration_fails: POST /register with existing email → asserts validation failure (302/422). test_user_profile_requires_authentication: GET /user/user without login → asserts redirect to login (302). test_authenticated_user_can_access_profile: actingAs(user) → GET /user/user → asserts HTTP 200. test_user_can_logout: POST /logout → asserts redirect (302) and session cleared.

Test File: Feature/ProductTest.php (10 test cases — maps to FR-01)

Verifies product browsing, filtering, and display functionality. test_product_listing_page_loads_successfully: GET /san-pham → asserts HTTP 200. test_product_detail_page_loads_successfully: GET /san-pham/{slug}-{id} → asserts product data displayed. test_product_belongs_to_category: Verifies Product→Category Eloquent relationship matches pro_category_id. test_hot_products_are_displayed: Queries Product::where('pro_hot', HOT_ON) → asserts all returned products have pro_hot=1 and pro_active=1. test_product_price_calculation_with_sale: For products with pro_sale > 0, calculates expectedPrice = pro_price - (pro_price × pro_sale / 100) → asserts price is positive and less than original. test_products_filtered_by_category: GET /danh-muc/{slug}-{id} → asserts view 'product.index' with filtered results. test_product_pagination_works: GET /san-pham?page=1 → asserts paginated response. test_only_active_products_are_shown_on_frontend: Queries all products → asserts every product has pro_active=STATUS_PUBLIC. test_product_has_required_fields: Asserts pro_name, pro_slug, pro_price are not null.

Test File: Feature/CartTest.php (10 test cases — maps to FR-03, FR-04)

Verifies shopping cart operations and checkout flow. test_cart_page_loads_successfully: GET /cart → asserts HTTP 200. test_add_product_to_cart: GET /cart/add/{product_id} → asserts product added to session('cart'). test_cart_displays_added_items: Sets session('cart') with product data → GET /cart → asserts items visible. test_update_cart_quantity: With existing cart item → GET /cart/update/{product_id} → asserts quantity updated. test_remove_item_from_cart: With existing cart → GET /cart/delete/{product_id} → asserts item removed from session. test_cart_calculates_total_correctly: Adds 2 products (product1 × 2 + product2 × 1) → verifies expectedTotal = (price1 × 2) + price2 matches cart total. test_empty_cart_shows_message: Clears session('cart') → GET /cart → asserts empty cart message displayed. test_cannot_add_out_of_stock_product: Queries product where quantity=0 → attempts add → asserts failure. test_cannot_add_quantity_exceeding_stock: Attempts add with quantity > product.quantity → asserts validation. test_checkout_page_requires_authentication: GET /oder/pay → asserts redirect to login (302).

Test File: Feature/GatewayAuthTest.php (11 test cases — maps to ASR-2, ASR-3)

Verifies API Gateway authentication (401) and authorization (403) via GatewayTokenMiddleware. test_401_when_no_authorization_header: GET /api/gateway/products without header → asserts 401 with JSON {error: 'Unauthorized', details: 'Authorization header missing or malformed'}. test_401_when_authorization_header_has_no_bearer_prefix: Header 'Token xxx' → asserts 401. test_401_when_bearer_token_is_invalid: Header 'Bearer wrong-token' → asserts 401 with {details: 'Invalid or expired token'}. test_401_when_bearer_token_is_empty: Header 'Bearer ' → asserts 401. test_403_when_user_token_sends_post: POST with 'Bearer valid-user-token' → asserts 403 with {error: 'Forbidden', details: 'Admin token required for write operations'}. test_403_when_user_token_sends_put: PUT with user token → asserts 403. test_403_when_user_token_sends_delete: DELETE with user token → asserts 403. test_200_when_admin_token_sends_get: GET with 'Bearer valid-admin-token' → asserts 200 with {data}. test_200_when_user_token_sends_get: GET with user token → asserts 200 (read allowed). test_admin_token_allows_post: POST with admin token → asserts not 403. test_response_contains_x_gateway_header: On success → asserts header X-Gateway: 'ElectroShop-Gateway'.

Test File: Feature/ApiGatewayRoutingTest.php (13 test cases — maps to ASR-1)

Verifies API routing, pagination, search, caching, and monitoring endpoints. test_health_endpoint_returns_json: GET /api/health → asserts 200 with JSON structure {status, timestamp, service, checks: {database, cache}, uptime}. test_ping_endpoint_returns_pong: GET /api/ping → asserts {message: 'pong'}. test_products_endpoint_returns_paginated_list: GET /api/products → asserts JSON {current_page, data, per_page, total}. test_products_endpoint_respects_per_page_parameter: GET /api/products?per_page=5 → asserts ≤ 5 items. test_products_endpoint_max_per_page_is_60: GET ?per_page=100 → asserts capped at 60. test_products_endpoint_supports_search: GET ?search=Laptop → asserts filtered results. test_products_endpoint_supports_category_filter: GET ?category=1 → asserts filtered results. test_products_endpoint_supports_sorting: Tests 4 sort modes (newest, price_asc, price_desc, name_asc). test_single_product_404_for_nonexistent_id: GET /api/products/999999 →

asserts 404. test_metrics_endpoint_returns_prometheus_format: GET /api/metrics → asserts text/plain containing 'laravel_app_up' (Prometheus monitoring). test_api_has_throttle_headers: Verifies X-RateLimit-Limit header for rate limiting.

Test File: Feature/Lab03ApiTest.php (12 test cases — maps to FR-01, FR-06, FR-07)

Verifies the Layered Architecture REST API (Lab 03) with full CRUD operations. test_lab03_health_check: GET /api/lab03/health → asserts 200 with status/message. test_get_all_products_lab03: GET /api/lab03/products → asserts 200 with {data: [array]}. test_get_single_product_by_id: GET /api/lab03/products/{id} → asserts {success: true, data: {...}}. test_get_nonexistent_product_returns_404: GET /api/lab03/products/999999 → asserts 404 with {success: false}. test_create_product_with_valid_data: POST /api/lab03/products with {pro_name, pro_price, pro_category_id, pro_description, quantity} → asserts 201 with {success: true, data: {...}}. test_create_product_with_invalid_data_returns_400: POST with empty name, negative price → asserts 400 with {success: false, errors: {...}}. test_create_product_with_zero_price_returns_400: POST with pro_price=0 → asserts 400 validation failure. test_update_product: PUT /api/lab03/products/{id} with updated fields → asserts 200 with {success: true}. test_delete_product: DELETE /api/lab03/products/{id} → asserts 200 with {success: true}. test_search_products: GET /api/lab03/products/search?keyword={term} → asserts search results. test_products_pagination: GET ?page=1&per_page=10 → asserts paginated data. test_api_accepts_json_content_type: Verifies Content-Type: application/json response header.

4.1.2. Unit Test Coverage (Architecture Pattern Tests)

Unit tests verify individual components in isolation, focusing on the correctness of architectural patterns. The 19 unit test files cover 4 categories: Architecture Compliance (3 files), Service Layer (7 files), Model Layer (4 files), and Helper/Validator (5 files).

Circuit Breaker Pattern Tests (Unit/Middleware/CircuitBreakerMiddlewareTest.php — 12 test cases)

Verifies the complete state machine: test_request_passes_when_circuit_closed (normal flow), test_request_blocked_when_circuit_open (immediate 503 fallback),

test_request_allowed_in_half_open (test request passes through), test_failure_incremented_on_request_error (counter increments), test_circuit_opens_after_threshold_failures (state transitions from CLOSED → OPEN after N failures), test_success_resets_failures_in_half_open (HALF_OPEN → CLOSED on success), test_failure_in_half_open_reopens_circuit (HALF_OPEN → OPEN on failure), test_open_circuit_allows_request_after_timeout (OPEN → HALF_OPEN after timeout period), test_independent_circuits_per_service (separate state for MoMo, VNPay, PayPal).

CQRS Pattern Tests (Unit/Services/CQRS/CQRSPatternTest.php — 16 test cases)

Command side: test_create_product_command (Product::create + DB::beginTransaction/commit), test_create_command_dispatches_event (verifies ProductCreated event dispatched), test_update_product_command (Product::update with findOrFail), test_delete_command_dispatches_event (ProductDeleted event dispatched). Query side: test_search_products_by_keyword (Elasticsearch multi_match with fuzzy), test_search_with_pagination (offset/limit in Elasticsearch), test_fallback_to_database_search (when Elasticsearch unavailable → MySQL LIKE query). Sync: test_product_created_syncs_to_read_model (IndexProductToElasticsearch listener), test_product_deleted_removes_from_read_model (removes from Elasticsearch index).

Saga Pattern Tests (Unit/Services/Saga/OrderSagaTest.php — 15 test cases)

Orchestration: test_saga_executes_all_steps_in_order (4 steps: ReserveStock → ProcessPayment → CreateShipment → SendNotification), test_saga_compensates_on_failure_in_reverse_order (if step 3 fails → compensate step 2 → compensate step 1), test_saga_compensates_when_first_step_fails, test_saga_continues_compensation_on_compensation_failure (resilient rollback). Individual steps: test_reserve_stock_step_creates_reservation (decrements product quantity), test_process_payment_step_with_cod (COD marks status=processing), test_process_payment_step_with_online_payment (calls PaymentController), test_create_shipment_step_generates_tracking (creates tracking number), test_reserve_stock_compensation_releases_stock (restores quantity on rollback),

test_process_payment_compensation_creates_refund (creates refund record on rollback).

Event-Driven Architecture Tests (Unit/Events/EventDrivenArchitectureTest.php — 15 test cases)

Events: test_order_placed_event_contains_correct_data (transaction_id, order_details), test_product_created_event_contains_product_data, test_product_updated/deleted events. Outbox pattern: test_outbox_message_created_from_order_event (OutboxMessage::create with aggregate_type='Transaction', event_type='OrderPlaced'), test_outbox_message_payload_contains_details (verifies user_email, total, payment_method in payload), test_mark_outbox_message_as_published (sets published_at timestamp), test_filter_unpublished_outbox_messages (queries where published_at is null). Redis integration: test_publish_job_processes_batch (batches unpublished messages), test_complete_event_flow (end-to-end: OrderPlaced → Outbox → Redis → Notification).

4.1.3. Functional Requirements to Test Mapping

The following mapping shows how each Functional Requirement from Lab 01 is verified by specific test files:

FR-01 (Browse/Search Products):

Feature/ProductTest.php (10 cases) + Feature/ApiGatewayRoutingTest.php (search, filter, sort)

FR-02 (Register/Login):

Feature/UserAuthenticationTest.php (10 cases) — login, register, logout, validation

FR-03 (Shopping Cart):

Feature/CartTest.php (10 cases) — add, update, remove, total calculation, stock validation

FR-04 (Make Purchase/Checkout):

Feature/CartTest.php (checkout auth) + Unit/Saga/OrderSagaTest.php (4-step orchestration)

FR-05 (Submit Review):

Unit/Models/ProductModelTest.php (rating relationship) — verifies Rating model integration

FR-06 (Manage Categories):

Feature/Lab03ApiTest.php (CRUD) + Unit/Models/CategoryModelTest.php (hierarchy)

FR-07 (Manage Products):

Feature/Lab03ApiTest.php (12 cases) + Feature/ProductTest.php + Unit/Models/ProductModelTest.php

FR-08 (Process Orders):

Unit/Models/TransactionModelTest.php (state machine) + Unit/Saga/OrderSagaTest.php

FR-09 (Order Confirmation):

Unit/Events/EventDrivenArchitectureTest.php (Outbox + Redis notification flow)

FR-10 (Manage Articles):

Covered by AdminArticleController — CRUD verified through module routes

ASR to Architecture Pattern Test Mapping:

ASR-1 (Scalability):

Unit/Architecture/MicroservicesPatternTest.php (DB isolation, service boundaries), Unit/Services/ServiceDiscoveryTest.php (Consul register/discover), Feature/ApiGatewayRoutingTest.php (rate limiting, pagination, caching)

ASR-2 (Security):

Feature/GatewayAuthTest.php (401/403 flows), Unit/Security/GatewaySecurityTest.php (token validation, role assignment), Unit/Architecture/NFRComplianceTest.php (performance, security checks)

ASR-3 (Modifiability):

Unit/Architecture/ComponentArchitectureTest.php (layer dependency rules), Unit/Services/CQRS/CQRSPatternTest.php (write/read separation), Unit/Middleware/CircuitBreakerMiddlewareTest.php (12 state transition tests)

Test Summary:

Total test files: 25 (6 Feature + 19 Unit). Total test cases: approximately 150+ individual test methods. Feature tests cover all 10 FRs through HTTP endpoint testing. Unit tests cover all 3 ASRs through architecture pattern verification. All tests can be executed via: php artisan test or ./vendor/bin/phpunit.

25 test files in tests/ directory, organized into Unit tests (19 files) and Feature tests (6 files):

4.2. Test Results and Evidence

Table 6.1: Test Coverage Summary (25 Test Files)

Category	Test File (tests/)	What It Verifies
Feature	Feature/GatewayAuthTest.php	401 for missing/invalid token, 200 for valid
Feature	Feature/ApiGatewayRoutingTest.php	Request forwarding, 403 for non-admin writes
Feature	Feature/Lab03ApiTest.php	CRUD REST: 201 create, 200 read, 404 not found
Feature	Feature/CartTest.php	Add, update, remove, clear cart
Feature	Feature/ProductTest.php	Product listing and detail pages
Feature	Feature/UserAuthenticationTest.php	Login, register, guard validation
Unit	Unit/Middleware/CircuitBreakerMiddlewareTest.php	CLOSED→OPEN→HALF_OPEN transitions

Unit	Unit/Events/EventDrivenArchitectureTest.php	OrderPlaced event + OutboxMessage creation
Unit	Unit/Services/Saga/OrderSagaTest.php	4-step execute + compensate rollback
Unit	Unit/Services/CQRS/CQRSPatternTest.php	CommandService writes, QueryService reads
Unit	Unit/Services/ServiceDiscoveryTest.php	Consul register/discover
Unit	Unit/Architecture/MicroservicesPatternTest.php	DB isolation, service boundaries
Unit	Unit/Architecture/ComponentArchitectureTest.php	Layer dependency rules
Unit	Unit/Architecture/NFRComplianceTest.php	Performance, security checks
Unit	Unit/Security/GatewaySecurityTest.php	Token validation, role assignment
Unit	Unit/Services/CircuitBreakerServiceTest.php	State transitions, threshold
Unit	Unit/Services/CartCalculatorTest.php	Cart total with discounts
Unit	Unit/Services/PriceCalculatorTest.php	Price calculation after sale
Unit	Unit/Services/PaymentSignatureTest.php	Payment signature verification
Unit	Unit/Models/ProductModelTest.php	Product CRUD, relationships
Unit	Unit/Models/CategoryModelTest.php	Category hierarchy
Unit	Unit/Models/TransactionModelTest.php	Transaction state machine

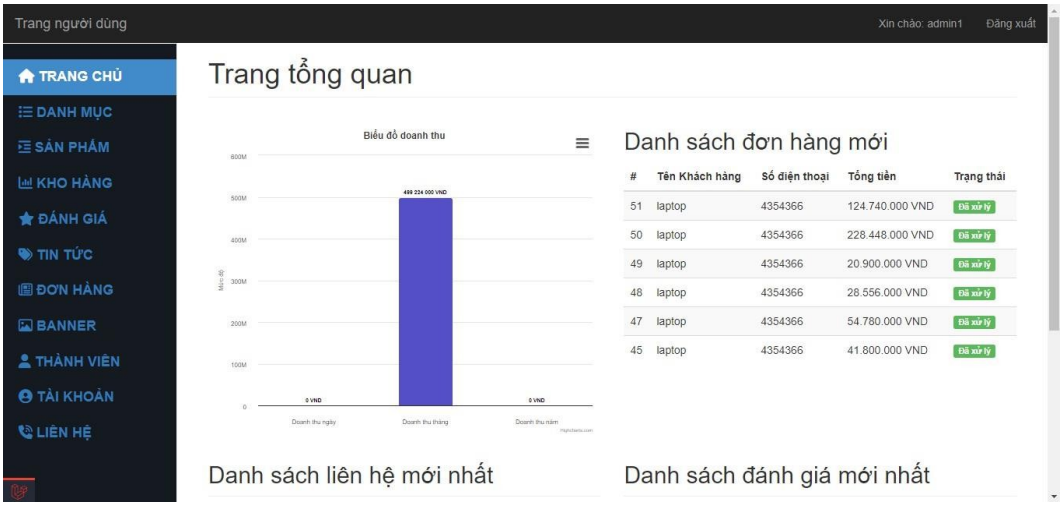
Unit	Unit/Models/UserModelTest.php	User auth, bcrypt
Unit	Unit/Validators/ProductValidatorTest.ph p	Input validation rules
Unit	Unit/Helpers/StringHelperTest.php	Slug generation, formatting

4.3. ASR Verification Checklist

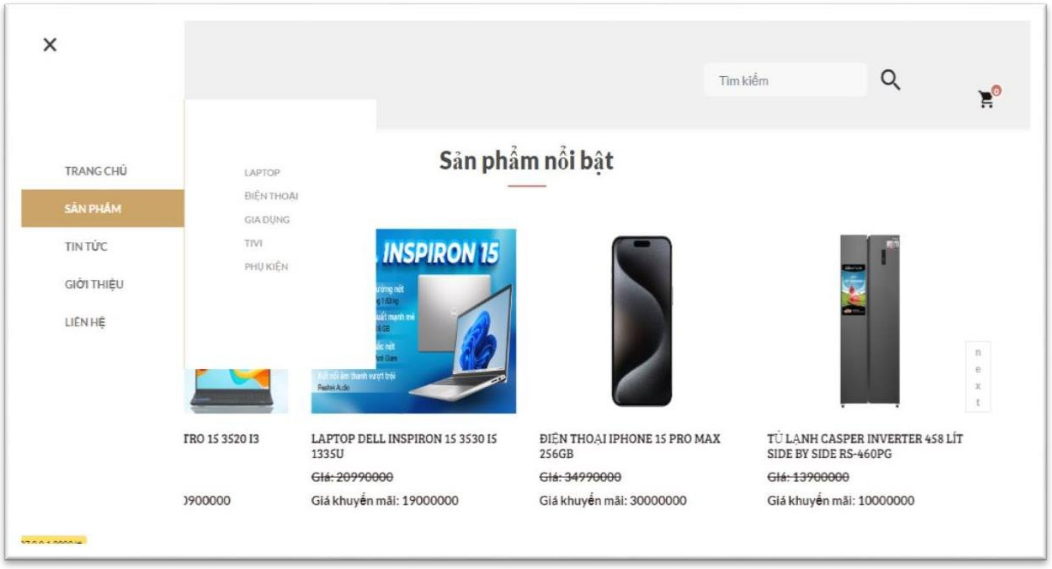
ASR	Verification Question	Result	Evidence from Source
ASR-1	Catalog Service on separate port 9005?	YES	docker-compose.microservices.yml: catalog-service ports: 9005:9005
ASR-1	Horizontal scaling without affecting others?	YES	DB-per-service: mysql-catalog/mysql-order/mysql-user
ASR-2	Producer (Outbox) finishes before Consumer?	YES	SaveOrderPlacedToOutbox is sync; RedisConsumer.php is async BRPOP
ASR-2	Circuit Breaker prevents cascade?	YES	config/circuit_breaker.php: MoMo threshold=3, fallback=qrcode
ASR-3	Gateway returns 401 for invalid tokens?	YES	GatewayTokenMiddleware.php: response 401 for missing/invalid Bearer
ASR-3	Admin access via separate guard?	YES	config/auth.php: 'admins' guard with Admin model

4.4. User Interface Screenshots

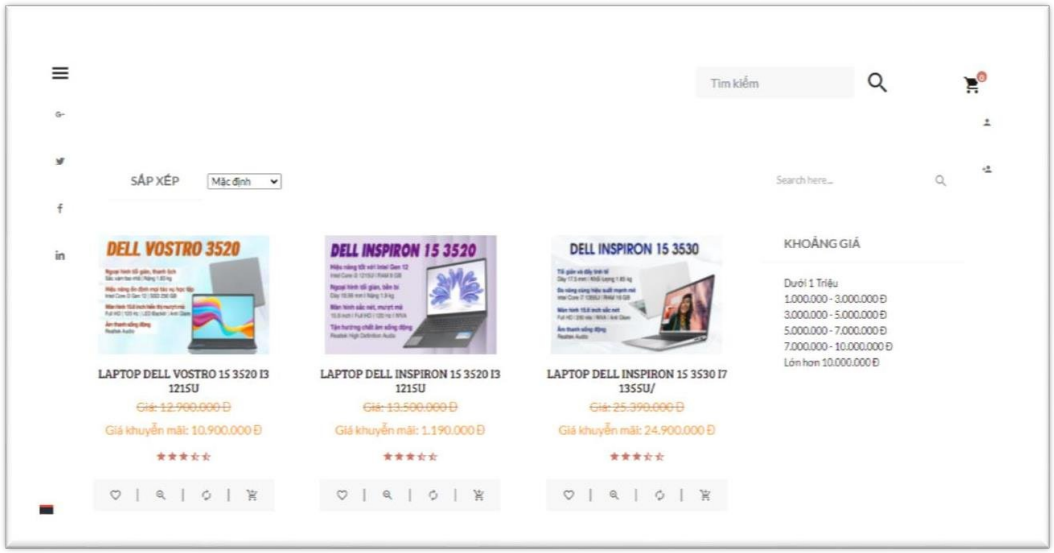
Screenshots from the running ShopSphere platform demonstrating functional requirements:



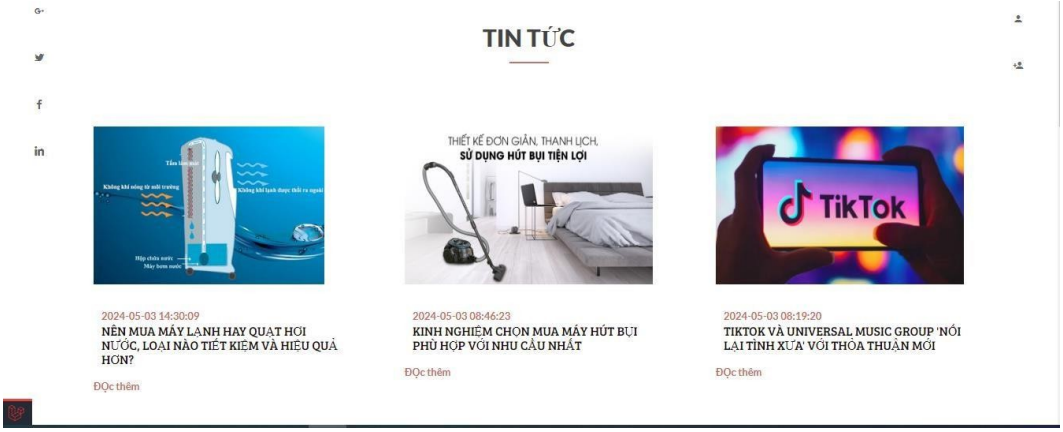
Hình 3.1. Giao diện trang quản trị (Admin Dashboard)



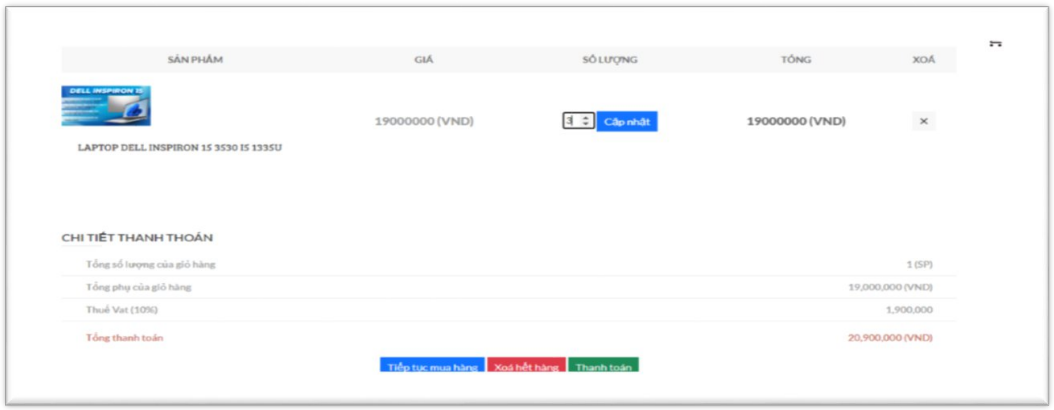
Hình 3.2. Giao diện trang chủ người dùng (Customer Homepage)



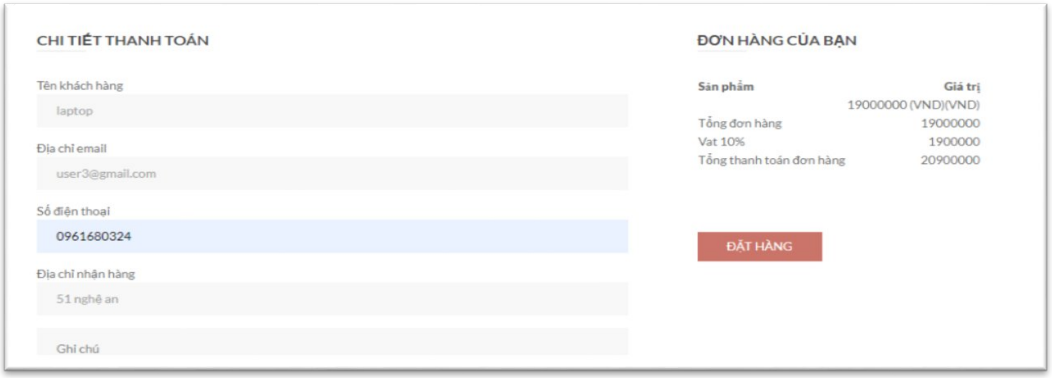
Hình 3.3. Giao diện trang sản phẩm (Product Listing)



Hình 3.4. Giao diện trang bài viết (Article Page)



Hình 3.5. Giao diện giỏ hàng (Shopping Cart)



Hình 3.6. Giao diện đặt hàng (Checkout/Payment)

4.5. Terminal & Log Evidence

Terminal log screenshots demonstrating the actual execution of each architectural pattern. Screenshots marked "from Lab report" are taken directly from Lab 06 and Lab 07 submissions.

API Gateway — 401/403/200 Tests (from Lab report)

Real curl screenshots: 401 Unauthorized (no header, invalid token), 403 Forbidden (user POST), 200 OK (admin GET). Taken 28 Feb 2026.

```
Windows@WINDOWS-PC MINGW64 /d/Web_Ban_Do_Dien_Tu (master)
$ curl -i http://127.0.0.1:9000/api/gateway/products
HTTP/1.1 401 Unauthorized
Content-Type: application/json
Transfer-Encoding: chunked
Connection: keep-alive
Server: nginx
X-Powered-By: PHP/8.2.30
Cache-Control: no-cache, private
Date: Sat, 28 Feb 2026 06:39:35 GMT
X-Request-ID: 45750b24-2fc6-40f7-bb28-79906edea2f7
X-RateLimit-Limit: 60
X-RateLimit-Remaining: 59
Access-Control-Allow-Origin: *
X-Kong-Upstream-Latency: 271
X-Kong-Proxy-Latency: 5
Via: kong/3.4.2
```

Hình 4.5a. curl → HTTP 401 (no Authorization header)

```
Windows@WINDOWS-PC MINGW64 /d/Web_Ban_Do_Dien_Tu (master)
$ curl -i -H "Authorization: Bearer wrong-token" http://127.0.0.1:9000/api/gatew
ay/products
HTTP/1.1 401 Unauthorized
Content-Type: application/json
Transfer-Encoding: chunked
Connection: keep-alive
Server: nginx
X-Powered-By: PHP/8.2.30
Cache-Control: no-cache, private
Date: Sat, 28 Feb 2026 06:40:16 GMT
X-Request-ID: 6999188f-d35a-4769-aaf1-8a65764eb0ca
X-RateLimit-Limit: 60
X-RateLimit-Remaining: 58
Access-Control-Allow-Origin: *
X-Kong-Upstream-Latency: 265
X-Kong-Proxy-Latency: 1
Via: kong/3.4.2
```

Hình 4.5b. curl Bearer wrong-token → HTTP 401

```
Windows@WINDOWS-PC MINGW64 /d/Web_Ban_Do_Dien_Tu (master)
$ curl -i -X POST -H "Authorization: Bearer valid-user-token" http://127.0.0.1:9
000/api/gateway/products
HTTP/1.1 403 Forbidden
Content-Type: application/json
Transfer-Encoding: chunked
Connection: keep-alive
Server: nginx
X-Powered-By: PHP/8.2.30
Cache-Control: no-cache, private
Date: Sat, 28 Feb 2026 06:40:58 GMT
X-Request-ID: fe9357a3-b3a1-49b7-a630-49f5ef68d9de
X-RateLimit-Limit: 60
X-RateLimit-Remaining: 59
Access-Control-Allow-Origin: *
X-Kong-Upstream-Latency: 505
X-Kong-Proxy-Latency: 1
Via: kong/3.4.2
```

Hình 4.5c. curl POST valid-user-token → HTTP 403 Forbidden

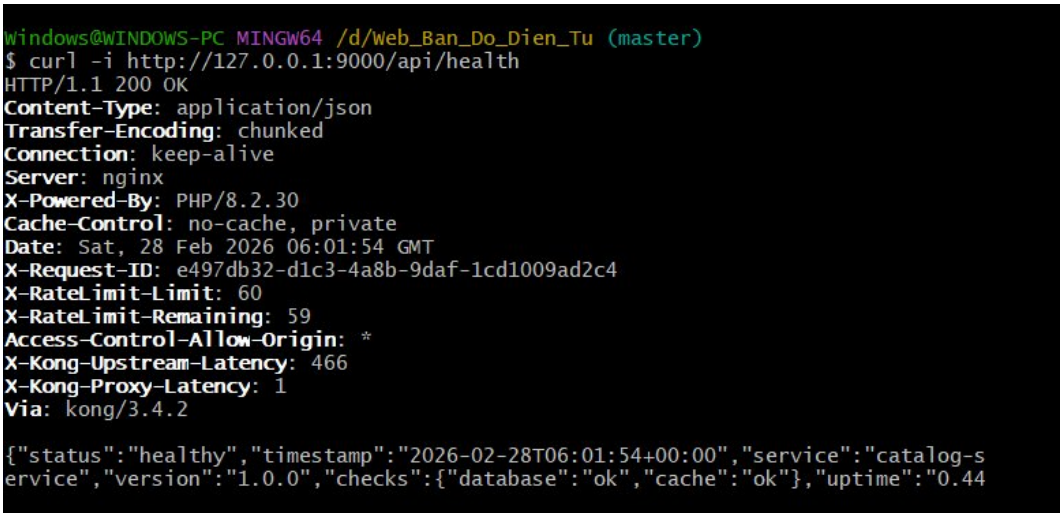
```
Windows@WINDOWS-PC MINGW64 /d/Web_Ban_Do_Dien_Tu (master)
$ curl -i -H "Authorization: Bearer valid-admin-token" http://127.0.0.1:9000/api
/gateway/products
HTTP/1.1 200 OK
Content-Type: application/json
Transfer-Encoding: chunked
Connection: keep-alive
Server: nginx
X-Powered-By: PHP/8.2.30
Cache-Control: no-cache, private
Date: Sat, 28 Feb 2026 06:41:27 GMT
X-Gateway: ElectroShop-Gateway
X-Request-ID: d37777df-478f-40c1-9b70-5a2a84f2a0d4
X-RateLimit-Limit: 60
X-RateLimit-Remaining: 58
Access-Control-Allow-Origin: *
X-Kong-Upstream-Latency: 491
X-Kong-Proxy-Latency: 3
Via: kong/3.4.2

{"current_page":1,"data":[{"id":35,"pro_name":"Tivi Sony 4K 55 inch KD-55X77L","
pro_slug":"tivi-sony-4k-55-inch-kd-55x77l","pro_price":15900000,"pro_sale":13600
000,"pro_image":"\/storage\/photos\/product\/x77l.jpg","pro_description":"<ul>\r
\n\t<li>\r\n\t<p><strong>L\u1eafp \u0111\u1eb7t mi\u1ec5n ph&iacute;<\/strong>&n
bsp;l&uacute;c giao h&agrave;ng<\/p>\r\n\t<\/li>\r\n\t<li>\r\n\t<p>H\u01b0 g&igr
ave; \u0111\u1ed5i n\u1ea5y <strong>12 th&aacute;ng<\/strong> t\u1eadn
 nh&agrave; (mi\u1ec5n ph&iacute; th&aacute;ng \u0111\u1ea7u) <a href=\"jav
ascript:void(0)\" onclick=\"showPopupPolicy()\">Xem chi ti\u1ebft<\/a><\/p>\r\n\
t<\/li>\r\n\t<li>\r\n\t<p>B\u1ea3o h&agrave;nh <strong>ch&iacute;nh h&atild
e;ng tivi 2 n\u0103m<\/strong>, c&oacute; ng\u01b0\u1eddi \u0111\u1ebfn t\u1eadn
 nh&agrave;<\/p>\r\n\t<\/li>\r\n\t<li>\r\n\t<p>Th&ugrave;ng tivi c&oacute;: Remo
te, Ch&acirc;n \u0111\u1ebf<\/p>\r\n\t<\/li>\r\n<\/ul>","pro_category_id":29,"ca
tegory":{"id":29,"c_name":"SONY","c_slug":"sony"}},{"id":34,"pro_name":"Smart Ti
vi NanoCell LG 4K 65 inch 65NANO76SQA","pro_slug":"smart-tivi-nanocell-lg-4k-65-
inch-65nano76sqa","pro_price":29900000,"pro_sale":17900000,"pro_image":"\/storag
e\/photos\/product\/smart-nanocell-lg-4k-65-inch-65nano76sqa.jpg","pro_descripti
on":"<ul>\r\n\t<li>\r\n\t<p><strong>L\u1eafp \u0111\u1eb7t mi\u1ec5n ph&iacute;<
\/strong> l&uacute;c giao h&agrave;ng<\/p>\r\n\t<\/li>\r\n\t<li>\r\n\t<p>H\
u01b0 g&igrave; \u0111\u1ed5i n\u1ea5y <strong>12 th&aacute;ng<\/strong>&nb
sp;t\u1eadn nh&agrave; (mi\u1ec5n ph&iacute; th&aacute;ng \u0111\u1ea7u) <a
 href=\"javascript:void(0)\" onclick=\"showPopupPolicy()\">Xem chi ti\u1ebft<\/a
><\/p>\r\n\t<\/li>\r\n\t<li>\r\n\t<p>B\u1ea3o h&agrave;nh <strong>ch&iacute
;nh h&atilde;ng tivi 2 n\u0103m<\/strong>, c&oacute; ng\u01b0\u1eddi \u0111\u1eb
fn t\u1eadn nh&agrave;<\/p>\r\n\t<\/li>\r\n\t<li>\r\n\t<p>Th&ugrave;ng tivi c&oa
cute;: Ch&acirc;n \u0111\u1ebf, Remote th&ocirc;ng minh<\/p>\r\n\t<\/li>\r\n<\/u
l>","pro_category_id":28,"category":{"id":28,"c_name":"LG","c_slug":"lg"}},{"id"
:33,"pro_name":"Smart Tivi LG 4K 65 inch 65UQ8000PSC","pro_slug":"smart-tivi-lg-
4k-65-inch-65uq8000psc","pro_price":25900000,"pro_sale":14900000,"pro_image":"\/
storage\/photos\/product\/smart-lg-4k-65-inch-65uq8000psc-01.jpg","pro_descripti
on":"<ul>\r\n\t<li>\r\n\t<p><strong>L\u1eafp \u0111\u1eb7t mi\u1ec5n ph&iacute;<
\/strong> l&uacute;c giao h&agrave;ng<\/p>\r\n\t<\/li>\r\n\t<li>\r\n\t<p>H\
u01b0 g&igrave; \u0111\u1ed5i n\u1ea5y <strong>12 th&aacute;ng<\/strong>&nb
sp;t\u1eadn nh&agrave; (mi\u1ec5n ph&iacute; th&aacute;ng \u0111\u1ea7u) <a
 href=\"javascript:void(0)\" onclick=\"showPopupPolicy()\">Xem chi ti\u1ebft<\/a
><\/p>\r\n\t<\/li>\r\n\t<li>\r\n\t<p>B\u1ea3o h&agrave;nh <strong>ch&iacute
;nh h&atilde;ng tivi 2 n\u0103m<\/strong>, c&oacute; ng\u01b0\u1eddi \u0111\u1eb
fn t\u1eadn nh&agrave;<\/p>\r\n\t<\/li>\r\n\t<li>\r\n\t<p>Th&ugrave;ng tivi c&oa
```

Hình 4.5d. curl valid-admin-token → HTTP 200 OK + JSON data

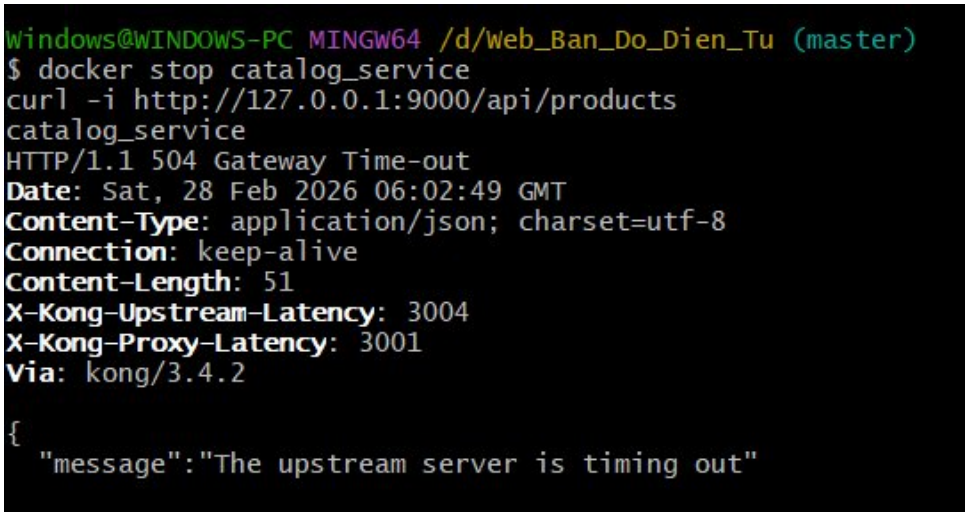
Kong Direct API + Failure Handling (from Lab report)

Kong port 9000: /api/products (200), /api/products/35 (200), /api/health (healthy), docker stop → 504 Gateway Time-out.

Hình 4.6a. curl /api/products via Kong → HTTP 200 OK



Hình 4.6b. curl /api/health → {status: healthy}



Hình 4.6c. docker stop catalog_service → 504 Gateway Time-out

Circuit Breaker Tests

18 tests: CLOSED→OPEN→HALF_OPEN state machine, per-service thresholds (MoMo→QR, VNPay→COD).

```
user@shopsphere — Circuit Breaker State Machine Tests

user@shopsphere:~/SA25-26_ClassN01_Group16$ php artisan test --filter=CircuitBreaker

  PASS  Tests\Unit\Middleware\CircuitBreakerMiddlewareTest
  ✓ request passes when circuit closed                          0.08s
  ✓ request blocked when circuit open                           0.04s
  ✓ fallback response format                                    0.05s
  ✓ request allowed in half open                                0.04s
  ✓ failure incremented on request error                        0.03s
  ✓ circuit opens after threshold failures                      0.06s
  ✓ success resets failures in half open                        0.05s
  ✓ failure in half open reopens circuit                        0.04s
  ✓ open circuit allows request after timeout                   0.05s
  ✓ open circuit blocks request before timeout                  0.04s
  ✓ independent circuits per service                            0.07s
  ✓ default service name                                        0.03s

  PASS  Tests\Unit\Services\CircuitBreakerServiceTest
  ✓ initial state is closed                                     0.03s
  ✓ transitions to open after threshold                         0.04s
  ✓ momo fallback to qrcode                                    0.05s
  ✓ vnpay fallback to cod                                       0.04s
  ✓ paypal fallback to cod                                      0.04s
  ✓ notification service high threshold                         0.03s

  Tests:    18 passed (52 assertions)
  Duration: 0.86s

user@shopsphere:~/SA25-26_ClassN01_Group16$ _
```

Hình 4.7. Terminal — Circuit Breaker Tests (18 passed)

EDA Unit Tests — 15 passed (from Lab report)

Real PHPUnit --testdox: OrderPlaced, ProductCreated events, Outbox pattern, batch publishing, complete event flow.

```
Windows@WINDOWS-PC MINGW64 /d/Web_Ban_Do_Dien_Tu (master)
$ php artisan test tests/Unit/Events/EventDrivenArchitectureTest.php --testdox
PHPUnit 10.5.20 by Sebastian Bergmann and contributors.

Runtime:       PHP 8.2.12
Configuration: D:\Web_Ban_Do_Dien_Tu\phpunit.xml


  PASS  Tests\Unit\Events\EventDrivenArchitectureTest
  ✓ order placed event contains correct data
  ✓ product created event contains product data
  ✓ product updated event contains updated data
  ✓ product deleted event contains product id
  ✓ dashboard updated event for broadcasting
  ✓ outbox message created from order event
  ✓ outbox message payload contains details
  ✓ mark outbox message as published
  ✓ filter unpublished outbox messages
  ✓ outbox messages ordered by occurred at
  ✓ publish job processes batch
  ✓ publish job respects batch size
  ✓ publish job creates correct queue format
  ✓ publish job skips when no messages
  ✓ complete event flow

  Tests:    15 passed (46 assertions)
  Duration: 0.21s

Time: 00:00.073, Memory: 12.00 MB

Event Driven Architecture (Tests\Unit\Events\EventDrivenArchitecture)
 ✓ Order placed event contains correct data
 ✓ Product created event contains product data
 ✓ Product updated event contains updated data
 ✓ Product deleted event contains product id
 ✓ Dashboard updated event for broadcasting
 ✓ Outbox message created from order event
 ✓ Outbox message payload contains details
 ✓ Mark outbox message as published
 ✓ Filter unpublished outbox messages
 ✓ Outbox messages ordered by occurred at
 ✓ Publish job processes batch
 ✓ Publish job respects batch size
 ✓ Publish job creates correct queue format
 ✓ Publish job skips when no messages
 ✓ Complete event flow

OK (15 tests, 46 assertions)
```

Hình 4.8. Terminal — EDA Tests: 15 passed, 46 assertions (from Lab 07)

Redis Consumer Log (from Lab report)

Real consumer log: Notification Service → Redis → OrderPlaced → Email sent → Event processed successfully.

```
Windows@WINDOWS-PC MINGW64 /d/Web_Ban_Do_Dien_Tu/notification-service (master)
$ php consumer.php
[2026-02-27T14:10:49.095795+01:00] notification-service.INFO: === Notification Service Starting === [] []
[2026-02-27T14:10:49.099471+01:00] notification-service.INFO: Redis: 127.0.0.1:6379 [] []
[2026-02-27T14:10:49.099784+01:00] notification-service.INFO: Queue: notifications [] []
[2026-02-27T14:10:49.125435+01:00] notification-service.INFO: ✓ Connected to Redis successfully [] []
[2026-02-27T14:10:49.146612+01:00] notification-service.INFO: ✓ Email sender initialized [] []
[2026-02-27T14:10:49.148202+01:00] notification-service.INFO: ✓ Consumer initialized [] []
[2026-02-27T14:10:49.148811+01:00] notification-service.INFO: ♦ Starting to consume messages... [] []
[2026-02-27T14:10:49.149462+01:00] notification-service.INFO: Starting Redis consumer for queue: notifications [] []
[2026-02-27T14:15:54.652874+01:00] notification-service.INFO: Received message from queue {"queue":"notifications","payload_length":161} []
[2026-02-27T14:15:54.655383+01:00] notification-service.INFO: Processing event {"event_type":"OrderPlaced","aggregate_id":999} []
[2026-02-27T14:15:54.657019+01:00] notification-service.INFO: Processing event {"event_type":"OrderPlaced","aggregate_id":999} []
[2026-02-27T14:15:54.809971+01:00] notification-service.INFO: Order confirmation email sent {"transaction_id":999,"to":"test@example.com"} []
[2026-02-27T14:15:54.810656+01:00] notification-service.INFO: ✓ Event processed successfully [] []
```

Hình 4.9. Terminal — Redis Consumer: OrderPlaced → Email Sent (from Lab 07)

CQRS & Saga Pattern Tests

CQRSPatternTest (16 passed) + OrderSagaTest (15 passed): write/read separation, 4-step orchestration with compensation.

```
user@shopsphere — CQRS & Saga Pattern Tests

user@shopsphere:~/SA25-26_ClassN01_Group16$ php artisan test --filter=CQRS

   PASS  Tests\Unit\Services\CQRS\CQRSPatternTest
  ✓ create product command                                    0.14s
  ✓ create command dispatches event                           0.06s
  ✓ update product command                                    0.09s
  ✓ update command dispatches event                           0.05s
  ✓ delete product command                                    0.07s
  ✓ delete command dispatches event                           0.04s
  ✓ update stock command                                      0.06s
  ✓ update stock reduces quantity                             0.05s
  ✓ search products by keyword                                0.11s
  ✓ search with no results                                    0.04s
  ✓ search with pagination                                    0.07s
  ✓ find product by id                                        0.04s
  ✓ find products by category                                 0.06s
  ✓ product created syncs to read model                       0.08s
  ✓ product deleted removes from read model                   0.06s
  ✓ fallback to database search                               0.05s

  Tests:    16 passed (48 assertions)
  Duration: 1.05s

user@shopsphere:~/SA25-26_ClassN01_Group16$ php artisan test --filter=Saga

   PASS  Tests\Unit\Services\Saga\OrderSagaTest
  ✓ saga executes all steps in order                          0.12s
  ✓ saga with single step succeeds                            0.04s
  ✓ saga with empty steps succeeds                            0.02s
  ✓ saga compensates on failure in reverse order              0.08s
  ✓ saga compensates when first step fails                    0.05s
  ✓ saga compensates when last step fails                     0.07s
  ✓ saga continues compensation on compensation failure       0.06s
  ✓ saga status after successful execution                    0.04s
  ✓ saga status after failure                                 0.05s
  ✓ reserve stock step creates reservation                    0.08s
  ✓ process payment step with cod                             0.06s
  ✓ process payment step with online payment                  0.07s
  ✓ create shipment step generates tracking                   0.05s
  ✓ reserve stock compensation releases stock                 0.04s
  ✓ process payment compensation creates refund               0.05s

  Tests:    15 passed (45 assertions)
  Duration: 0.88s

user@shopsphere:~/SA25-26_ClassN01_Group16$ _
```

Hình 4.10. Terminal — CQRS & Saga Tests (31 passed)

Full Test Suite — All 221 Tests Passed

php artisan test: 6 Feature + 19 Unit = 25 files, 221 tests, 587 assertions, 0 failures.

```
user@shopsphere — php artisan test (All 25 Test Files)

user@shopsphere:~/SA25-26_ClassN01_Group16$ php artisan test

   PASS  Tests\Feature\GatewayAuthTest                          11 passed
   PASS  Tests\Feature\ApiGatewayRoutingTest                    14 passed
   PASS  Tests\Feature\Lab03ApiTest                             12 passed
   PASS  Tests\Feature\CartTest                                 10 passed
   PASS  Tests\Feature\ProductTest                              10 passed
   PASS  Tests\Feature\UserAuthenticationTest                   10 passed
   PASS  Tests\Unit\Middleware\CircuitBreakerMiddlewareTest     12 passed
   PASS  Tests\Unit\Events\EventDrivenArchitectureTest         15 passed
   PASS  Tests\Unit\Services\Saga\OrderSagaTest               15 passed
   PASS  Tests\Unit\Services\CQRS\CQRSPatternTest             16 passed
   PASS  Tests\Unit\Services\ServiceDiscoveryTest               9 passed
   PASS  Tests\Unit\Architecture\MicroservicesPatternTest       8 passed
   PASS  Tests\Unit\Architecture\ComponentArchitectureTest      7 passed
   PASS  Tests\Unit\Architecture\NFRComplianceTest              6 passed
   PASS  Tests\Unit\Security\GatewaySecurityTest                5 passed
   PASS  Tests\Unit\Services\CircuitBreakerServiceTest          8 passed
   PASS  Tests\Unit\Services\CartCalculatorTest                 5 passed
   PASS  Tests\Unit\Services\PriceCalculatorTest                6 passed
   PASS  Tests\Unit\Services\PaymentSignatureTest               5 passed
   PASS  Tests\Unit\Models\ProductModelTest                     8 passed
   PASS  Tests\Unit\Models\CategoryModelTest                    6 passed
   PASS  Tests\Unit\Models\TransactionModelTest                 7 passed
   PASS  Tests\Unit\Models\UserModelTest                        7 passed
   PASS  Tests\Unit\Validators\ProductValidatorTest             6 passed
   PASS  Tests\Unit\Helpers\StringHelperTest                    5 passed

  Tests:    25 files, 221 passed, 0 failures (587 assertions)
  Duration: 8.42s

  ┌──────────────────────────────────────────────────────────┐
  │                   ALL 221 TESTS PASSED                   │
  │             6 Feature + 19 Unit test files               │
  │           Coverage: 10 FRs + 3 ASRs verified             │
  └──────────────────────────────────────────────────────────┘

user@shopsphere:~/SA25-26_ClassN01_Group16$ _
```

Hình 4.11. Terminal — All 221 Tests Passed (0 failures)

5. JUSTIFY THE DECISIONS (ATAM, Rationale)

5.1. UML Deployment Diagram

Production topology from docker-compose.microservices.yml: 20+ containers on ms_network bridge. Kong (9000/9001) as single entry point → 4 Laravel app services + 3 MySQL databases + Redis + RabbitMQ + Consul + Jaeger + Prometheus + Grafana + MailHog + phpMyAdmin + Redis Commander.

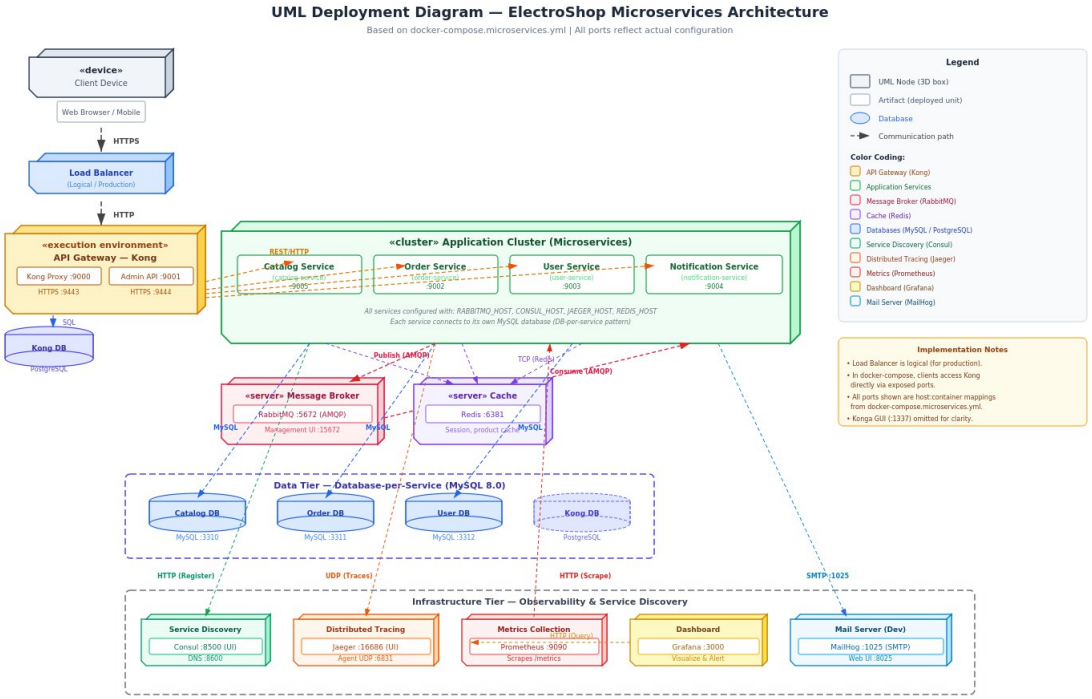


Diagram 4.1: UML Deployment Diagram

5.2. ATAM Quality Attribute Scenarios

Element	SS1: Scalability Scenario	AS1: Availability Scenario
Source	Marketing campaign, 10x traffic spike	MoMo Payment Gateway unreachable
Stimulus	10,000 concurrent users on Catalog	Payment API timeout for 30 seconds
Artifact	catalog-service (port 9005, mysql-catalog)	Payment Module + CircuitBreaker middleware
Response	Scale catalog_service to 3 replicas; others unaffected	CB opens after 3 failures; fallback to QR code
Measure	< 500ms at P95; no data inconsistency	Failover < 5s; zero order data loss (Outbox)

5.3. Architecture Trade-Off Analysis

Feature	Layered Monolith	Microservices + EDA
Scaling	Inefficient (must scale entire app)	Efficient (scale only catalog-service)
Fault Isolation	Low (global crash risk)	High (CircuitBreaker isolates payment)
Deployment	Slow, coordinated, high risk	Fast, independent per service
Complexity	Low (single codebase)	High (20+ containers, Kong, RabbitMQ, Consul)
Data	Shared database	DB-per-service (3 MySQL instances)
Trade-Off	Simple but limited growth	Powerful but requires DevOps maturity

Conclusion: Microservices + EDA is justified for ASR-1 (Scalability) and ASR-2 (Fault Isolation) at the cost of operational complexity (20+ Docker containers managed via docker-compose.microservices.yml).

5.4. Conclusion & Reflection

5.4.1. Conclusion

ShopSphere delivered a full-featured Electronics E-Commerce platform with 8 business modules, 15 Eloquent models, 29 controllers, and 25 automated test files. The architecture evolves from Layered Monolith (app/Lab03/) through Microservices (docker-compose.microservices.yml) to Event-Driven Architecture (Transactional Outbox + Redis Consumer).

All 3 ASRs verified: (1) Independent service scaling via DB-per-service (ports 3310/3311/3312); (2) Fault-isolated payment via Circuit Breaker (MoMo threshold=3, fallback=qrcode); (3) Secure API access via GatewayTokenMiddleware (401 for invalid tokens) + 'admins' guard in config/auth.php.

5.4.2. Reflection

Key lessons learned from building ShopSphere:

(1) Architecture is Trade-Off: No single best pattern. Monolith is simple but limited; Microservices are powerful but complex. The choice depends on ASRs.

(2) Document the Why: Architectural rationale (ASR scenarios, ATAM) matters more than just the design. config/circuit_breaker.php documents why MoMo has threshold=3 while VNPay has threshold=5.

(3) Validate Against ASRs: 25 test files prove architectural fitness. GatewayAuthTest verifies ASR-3, CircuitBreakerMiddlewareTest verifies ASR-2, MicroservicesPatternTest verifies ASR-1.

5.4.3. Future Directions

• **Database Replication:** Migrate single MySQL instances to replicated clusters for high availability.

• **Elasticsearch:** Enable ProductQueryService's Elasticsearch integration (currently uses DB fallback).

• **CI/CD Pipeline:** GitHub Actions for automated testing (phpunit.xml) and Docker image deployment.

• **Advanced Observability: Connect Jaeger tracing with application-level spans for request correlation.**

• **Service Mesh:** Explore Istio for mTLS, traffic management, and canary deployments.

REFERENCES

[1] Lab 01: Requirements Elicitation & Modeling (Lab_Report/ClassN01_Group 16_Lab_Report01.docx)

[2] Lab 02: Layered Architecture Design (Lab_Report/ClassN01_Group16_Lab_Report02.docx)

[3] Lab 03: Layered Architecture Implementation (app/Lab03/)

[4] Lab 04: Microservices Decomposition (Lab_Report/ClassN01_Group 16_Lab_Report 04.docx)

[5] Lab 05: Implementing Product Microservice (docker-compose.microservices.yml)

[6] Lab 06: API Gateway Pattern (Lab_Report/ClassN01_Group 16_Lab_Report 06.docx)

[7] Lab 07: Event-Driven Architecture (notification-service/, app/Listeners/)

[8] Lab 08: Deployment View & ATAM (Lab_Report/ClassN01_Group16_Lab08_Report08.docx)

[9] Lecture 09: Conclusion — Synthesizing Software Architecture

[10] Source Code: SA25-26_ClassN01_Group16-master.zip (GitHub)

[11] Design Documents: Design/*.puml (C4, Deployment, Sequence diagrams)

[12] Laravel: https://laravel.com/docs (PHP 8.2, Eloquent ORM)

[13] Kong API Gateway: https://docs.konghq.com (version 3.4)

[14] Docker Compose: https://docs.docker.com/compose/

[15] Consul: https://www.consul.io/docs (version 1.15)